TileFlow: Spatial Neural Networks on AMD XDNA 2 NPU

Hardware-software co-design for close-to-metal neural network inference

Built with IRON/MLIR-AIE toolchain — Target: AMD Ryzen AI 9 HX 370 (Strix Point)

1. Introduction

Modern neural processing units (NPUs) are spatial dataflow computers: instead of a single CPU with caches and a register file, they expose a 2D array of small compute tiles, each with its own SRAM and SIMD units, connected by a programmable interconnect. This architecture is extremely efficient for *on-chip* computation — but only if the software maps the algorithm directly to the physical hardware.

TileFlow takes this literally. We design a neural network whose architecture is dictated by the physical tile layout of the AMD XDNA 2 NPU. The network has learnable parameters and non-linearities (ReLU), but its structure (number of layers, block grouping, buffer sizes) matches the hardware exactly.

Our main result is a **block-recurrent character language model** that generates Shakespeare-like text entirely on the NPU. The model has 32 layers grouped into 8 blocks of 4. Each block maps to a 4-stage pipeline on the NPU (one layer per compute row, 8 columns in parallel). Between blocks, the CPU applies normalisation, character-embedding injection, and residual connections — the same operations the model uses during training. The result: **89,600 characters/second** on 32 NPU tiles, with the trained weights producing *identical* computation on hardware — no approximations, no graph compilation, no operator fallback.

> **Key principle:** We design the network to match the hardware, not the other way around. Any architecture with learnable parameters and non-linearities can learn — so we choose the one that maximizes hardware utilization.

2. Background

This section provides the hardware and systems context that most ML engineers never need to think about — until they want to understand *why* certain hardware is fast (or slow) for their

models. If you've trained models with PyTorch or JAX and think of hardware as "a GPU with some VRAM," this section fills in the gap between that abstraction and the physical reality of a spatial processor like the XDNA 2 NPU.

2.1 Glossary of acronyms

The table below defines every acronym used in this paper. Each entry has an `id` so that occurrences in the text link back here. We group them by domain so you can revisit this section as a reference while reading.

Acronym	Stands for	What it means
Hardware & memory		
NPU	Neural Processing Unit	A dedicated accelerator for neural-network inference (and sometimes training), built into a laptop or phone chip.
APU	Accelerated Processing Unit	AMD's name for a single die that integrates CPU + GPU + NPU.
XDNA	(AMD brand name)	AMD's NPU architecture family. XDNA 2 is the second generation, found in "Strix Point" Ryzen AI chips.
AIE	AI Engine	The individual tile processor IP inside the XDNA NPU, originally designed by Xilinx (acquired by AMD).
SRAM	Static Random-Access Memory	Fast, on-chip memory (~1–2 ns access). Each compute tile has ~64 KB of SRAM. Expensive per bit, but extremely fast because it sits right next to the compute logic.
DRAM	Dynamic Random-Access Memory	The main system memory (8–64 GB). Much slower than SRAM (~50–100 ns access) but vastly cheaper per bit.
DDR	Double Data Rate	The interface standard for DRAM modules. "DDR memory" is the system RAM your laptop uses. In this paper, "DDR" means "host-side main memory."
DMA	Direct Memory Access	A hardware mechanism that copies data between memory regions *without using the CPU*. The NPU's shim tiles contain DMA engines that move data between DDR and tile SRAM.
PCIe	Peripheral Component Interconnect Express	The high-speed bus connecting the NPU to the rest of the system.
Compute concepts		

SIMD	Single Instruction, Multiple Data	A processor executes one instruction on a *vector* of values simultaneously. For example, multiplying 32 numbers in one clock cycle instead of one at a time. This is how GPUs and NPUs achieve massive throughput.
VLIW	Very Long Instruction Word	A processor design where each instruction encodes *multiple operations* to execute in parallel (e.g., a multiply, an add, and a load all in one cycle). Each AIE tile uses a VLIW core.
MMUL	Matrix Multiply (unit)	A hardware block dedicated to multiplying small matrices (e.g., 8×8 blocks of bfloat16). This is the workhorse of each AIE tile.
FIFO	First In, First Out	A queue where data is read in the same order it was written. The NPU uses hardware FIFOs ("ObjectFIFOs") to stream data between tiles, like Unix pipes between processes.
GEMM	General Matrix Multiply	The standard linear algebra operation C = A × B + C. Neural network layers are essentially sequences of GEMMs with non-linearities in between.
Numeric formats		
bf16 / bfloat16	Brain Floating Point, 16-bit	A 16-bit floating-point format with the same exponent range as float32 (8 bits) but less precision (7-bit mantissa vs 23-bit). Invented at Google Brain for ML training where range matters more than precision.
BFP16	Block Floating Point, 16-bit	An emulation mode on AIE2P that groups bf16 values into blocks sharing a common exponent. Enables efficient SIMD matmul in the MMUL unit with tile factor r = s = t = 8.
INT8	8-bit Integer	8-bit integer arithmetic, used for quantized inference. The NPU's peak in INT8 mode is 50 TOPS (double the bf16 rate).
Performance metrics		
FLOPS	Floating-point Operations Per Second	The standard measure of compute throughput. One multiply-add on two numbers counts as 2 FLOPs.
GFLOPS	Giga FLOPS (10^9)	Billions of floating-point operations per second.
TFLOPS	Tera FLOPS (10^{12})	Trillions of floating-point operations per second. The NPU's peak is 25 TFLOPS in bfloat16.
TOPS		

	Tera Operations Per Second	Like TFLOPS but for integer operations. Used for INT8 peak specs (50 TOPS for this NPU).
Machine learning		
MLP	Multi-Layer Perceptron	A neural network made of fully-connected (dense) layers. Each layer computes y = activation(x × W + b).
ReLU	Rectified Linear Unit	The activation function max(x, 0). Simple, cheap to compute, and widely used.
RMSNorm	Root Mean Square Normalisation	A layer-normalisation variant that divides each vector by its RMS value: x / sqrt(mean(x^2) + ε). Cheaper than LayerNorm because it skips mean subtraction.
Toolchain & software		
IRON	(name, not an acronym)	AMD's Python API for programming AIE tiles at a high level: defining kernels, ObjectFIFOs, and tile placements.
MLIR	Multi-Level Intermediate Representation	A compiler framework (from the LLVM project) that represents programs at multiple abstraction levels. MLIR-AIE is the dialect that targets AIE hardware.
XRT	Xilinx Runtime	The userspace library and kernel driver that loads bitstreams onto the NPU and manages execution.
XCLBIN	Xilinx Container for Linux Binary	The compiled binary file that contains the NPU bitstream (tile configuration, routing, kernel code).
LLVM	Low Level Virtual Machine	A widely-used compiler infrastructure. Peano/LLVM-AIE is a fork that compiles C++ to AIE tile machine code.
BD	Buffer Descriptor	A hardware structure in the DMA engine that describes one data transfer: source address, size, stride pattern. The 10-bit size field (max 1024) is a constraint discussed in Section 3.6.

2.2 Key vocabulary (non-acronyms)

Several important terms in this paper are not acronyms but may be unfamiliar to readers who have worked above the hardware abstraction layer:

Term	What it means
Shim tile	The row of interface tiles at the bottom of the NPU array (row 0). "Shim" means a thin adapter layer — these tiles *bridge* two different worlds: the system's DDR memory (accessed via PCIe/AXI bus) and the NPU's

	internal tile interconnect. Each shim tile contains DMA engines that translate between host memory addresses and on-chip tile coordinates. They do no computation; they are pure data movers.
Tile	A self-contained processing unit with its own local memory (SRAM), instruction memory, and compute logic. Unlike CPU cores that share caches, each tile is an *independent computer* that communicates with neighbours through explicit data channels (FIFOs), not shared memory.
Bitstream	The compiled binary that programs the NPU's hardware configuration: which tiles run which code, how the interconnect routes data, and what DMA transfers to perform. Analogous to a GPU shader binary, but configuring a full spatial processor rather than a single shader core. Packaged in an XCLBIN file.
Kernel (NPU)	A C++ function compiled to run on a single tile. Not to be confused with an OS kernel or a CUDA kernel (which runs on thousands of GPU threads). An NPU kernel runs on exactly *one* tile's VLIW+SIMD core and has direct access to that tile's SRAM.
Pipeline	A technique where multiple stages of a task overlap in time, like an assembly line. While stage N processes item *i*, stage N+1 processes item *i–1*. The chess compiler pipelines the inner loops of our matmul kernel so that memory loads, multiplications, and stores overlap across iterations.
Register	The smallest, fastest storage inside a processor core — a few hundred bytes at most, accessible in a single clock cycle. The MMUL unit accumulates partial matrix products in registers before writing back to SRAM. Our fused kernel zero-initialises these accumulators in registers, avoiding a separate SRAM clear pass.
Memory wall	The growing gap between processor speed and memory speed. CPUs can execute arithmetic ~100× faster than DRAM can supply data. This is *the* fundamental reason NPUs (with data-local SRAM) can be dramatically faster than CPUs for the right workloads.
MemTile	A memory-only tile (no compute core) in the NPU array, sitting between the shim row and the compute rows. Contains 512 KB of SRAM and powerful DMA engines that can split, forward, or join data streams, routing data from the shim to multiple compute rows.
Interconnect	The on-chip wiring and switches that connect tiles to each other and to memory tiles. Unlike a CPU's shared bus, the NPU interconnect is *circuit-switched*: routes are configured at compile time, giving dedicated bandwidth with no contention.

Throughput vs Latency	**Latency** is how long one operation takes (e.g., one matmul = 1 μs). **Throughput** is how many operations complete per second (e.g., 24 tiles × 1000 matmuls = 24M matmuls/s). The NPU excels at throughput through massive parallelism, even if single-tile latency is similar to a CPU.
Double buffering	Using two buffers that swap roles each iteration: one is being read by the compute unit while the other is being filled by DMA. Also called "ping-pong buffering." This hides data transfer latency behind computation.

2.3 How CPUs execute neural networks (and why it's slow)

When you call `y = torch.relu(x @ W)` on a CPU, here is what actually happens at the hardware level:

1. The weight matrix `W` lives in DRAM. The CPU issues a load instruction, and the data travels through the **memory hierarchy**: DRAM → L3 cache → L2 cache → L1 cache → registers.
2. The CPU's SIMD units multiply a few rows at a time. The result goes back through the cache hierarchy to DRAM.
3. For the next layer, the process repeats: read the activation from DRAM, read the next weight matrix from DRAM, compute, write back.

The fundamental problem is the **memory wall**: modern compute units can perform arithmetic far faster than memory can supply data. A typical CPU can do ~1 TFLOPS of bf16 math, but its DDR bandwidth is ~50 GB/s. To keep the ALUs busy doing a 128×128 matmul, you need 128×128×2 = 32 KB of weight data delivered every ~1 μs — which is 32 GB/s just for one matrix. If you're running 4 layers, that's 128 GB/s, already exceeding the memory bandwidth. The CPU stalls waiting for data.

> **The memory wall in one sentence:** Arithmetic is cheap; moving data is expensive. The deeper you go in a neural network, the more time is spent shuffling data between memory levels, not doing useful math.

2.4 The spatial dataflow alternative

A **spatial architecture** takes a radically different approach. Instead of one fast processor with a deep cache hierarchy, it uses *many small processors* (tiles), each with a tiny but *extremely fast* local memory (SRAM). Data moves between tiles through dedicated hardware channels (FIFOs), not through shared caches.

Think of it like a factory assembly line versus a single master craftsman:

- **CPU (craftsman):** One highly skilled worker goes to the warehouse (DRAM) for each part, brings it to the workbench (registers), processes it, takes the result back to the warehouse, gets the next part. Most time is spent walking, not working.
- **NPU (assembly line):** Many workers sit at their stations with all their parts already on their desk (SRAM). A conveyor belt (FIFO) moves the workpiece from one station to the next. Nobody walks anywhere.

The key property is **data locality**: once data is loaded into a tile's 64 KB SRAM, it stays there for as many operations as you can do on it. There is no cache to get evicted from, no bus to contend for. If a 128×128 weight matrix (32 KB) fits in SRAM, you can multiply against it thousands of times at full speed — which is exactly what our recurrent MLP does.

2.5 Key hardware concepts used in this project

These concepts appear throughout the paper. You don't need to understand every transistor, but knowing these ideas will make the design choices clear:

Double buffering (ping-pong)

If a tile needs to *receive* new data while *computing* on data it already has, you need two buffers: one being filled by DMA while the other is being read by the compute unit. They swap roles each iteration. We use this for the activation buffers: buffer A holds the input, the tile computes into buffer B, then B becomes the input and A becomes the output.

Tiled matrix layout

A "row-major" matrix stores elements left-to-right, top-to-bottom: `[[a, b], [c, d]]` becomes `[a, b, c, d]`. The AIE matmul unit instead expects a **blocked (tiled) layout**: the matrix is divided into 8×8 sub-matrices, and each block is stored contiguously. This matches the MMUL hardware which multiplies 8×8 blocks in one operation. The `to_tiled()` function handles this conversion.

ObjectFIFOs and data movement

On a CPU, data movement is implicit — you `load` from an address and the cache hierarchy handles the rest. On the NPU, you must *explicitly* program every data transfer: "move 4 KB from DDR address X to tile (3, 2)'s input buffer." IRON's `ObjectFifo` abstraction makes this manageable: you declare a typed channel between a producer and a consumer, and the compiler generates the DMA configurations.

The invocation overhead problem

Every time the host CPU tells the NPU to run, there is a fixed overhead of ~120 μs for driver calls, instruction dispatch, and DMA setup. This is analogous to the overhead of launching a CUDA kernel on a GPU. If your actual compute takes only 1 μs (as it does for a single small matmul), you are spending 99% of the time on overhead. The solution is to do *lots of work per invocation* — hence the hardware loop that repeats thousands of matmuls before returning to the host.

2.6 How to read the rest of this paper

With this background, the rest of the paper should be accessible:

- **Section 3** is the NPU primer: physical hardware, tile microarchitecture, the C++ kernel API, the IRON Python programming model, design patterns, and hardware constraints.
- **Section 5** presents the neural network architecture: block-recurrent character language model mapped to the NPU pipeline.
- **Section 6** shows quality and throughput results.
- **Section 7** describes the software toolchain.
- **Section 8** maps the code structure to the concepts.

2.7 Neural network fundamentals (for hardware engineers)

If you work in hardware or systems and have heard “neural network” but never implemented one, this section gives you just enough to follow the architecture decisions in this paper.

What is a neural network?

A neural network is a function built by stacking **layers**. Each layer applies a linear transformation (matrix multiply) followed by a **non-linearity** (a simple element-wise function). For example:

```
y = ReLU(x @ W + b)      # one layer
```

Here `x` is the input vector, `W` is a learnable **weight matrix**, `b` is a learnable **bias vector**, and ReLU(z) = max(z, 0) is the non-linearity. Without the non-linearity, stacking layers would collapse to a single matrix multiply — the non-linearity is what gives depth its power. Stacking many such layers creates a “deep” network that can learn increasingly abstract representations of its input.

Language models: predicting the next character

A **language model** assigns probabilities to sequences of text. The simplest version predicts the next character (or word) given everything that came before. If the model has read “KING RICHAR”, it should assign high probability to “D”. At generation time, we sample from this probability distribution to produce text one character at a time.

The quality of a language model is measured by **cross-entropy loss** — roughly, how surprised the model is by each character in a held-out test set. Lower is better. The related metric **perplexity** = e^{loss} can be interpreted as “the model is as confused as if choosing uniformly among *N* options.” A perplexity of 11 means the model is, on average, as uncertain as if it were picking among 11 equally likely characters (out of ~65 in our vocabulary).

Recurrence: memory across time

A **recurrent** network maintains a **hidden state** `h` that carries information from one time step to the next:

```
h = f(h, embed(char))     # update hidden state with new character
logits = h @ W_out + b    # predict next character from hidden state
```

The function `f` reads the previous hidden state and the current character’s **embedding** (a learned vector that represents each character as a point in continuous space), and produces a new hidden state. This is fundamentally different from feedforward networks that process each input independently: the hidden state acts as the model’s “memory” of everything it has read so far.

Making deep networks trainable

Simply stacking 32 layers of `h = ReLU(h @ W + b)` does not work: the hidden state either explodes to infinity or collapses to zero as it passes through many layers. Three techniques solve this:

- **Residual connections:** Instead of `h = f(h)`, compute `h = h + f(h)`. The identity “shortcut” ensures that gradients can flow backwards through the network even if `f` has very small gradients. This was the key innovation behind deep learning’s leap from ~10 to ~100+ layer networks.
- **Normalisation (RMSNorm):** Before each layer, rescale the hidden state to have unit root-mean-square: `h / sqrt(mean(h²) + ε)`. This prevents the activations from growing exponentially across layers or time steps.

- **Input injection:** Add the character embedding at every layer (not just the first), so each layer can directly access the current input rather than relying on information that has been progressively transformed and potentially diluted by many preceding layers.

Training: learning by gradient descent

The weight matrices start with random values. **Training** means repeatedly: (1) run the model on a batch of text, (2) compute how wrong the predictions are (the loss), (3) compute the gradient of the loss with respect to every weight (using **backpropagation** — the chain rule applied systematically through the network), and (4) nudge each weight in the direction that reduces the loss. For recurrent networks, this is called **backpropagation through time (BPTT)** because gradients flow backwards through both layers and time steps.

Training typically happens on a GPU because GPUs have massive memory bandwidth and can process large batches in parallel. The NPU enters at **inference** time: once the weights are learned, we deploy the trained model to the NPU for fast, efficient text generation.

3. The Hardware

The AMD XDNA 2 NPU in the Ryzen AI 9 HX 370 (codename Strix Point) is a tiled spatial-dataflow processor with the following structure:

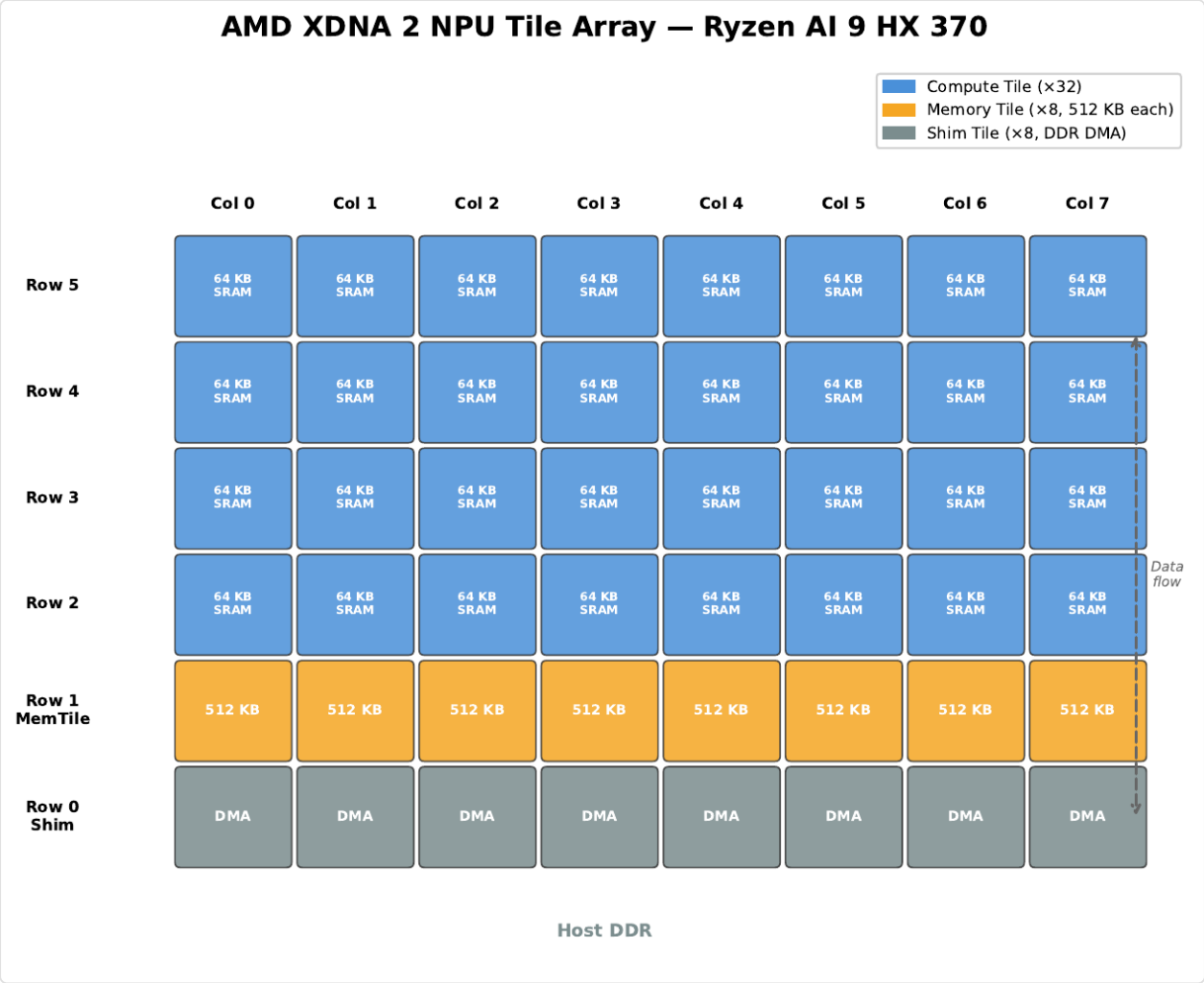

Figure 1: Physical tile array of the AMD XDNA 2 NPU. 32 compute tiles (rows 2–5) each contain ~64 KB SRAM and a bf16 MMUL unit. 8 memory tiles (row 1, 512 KB each) serve as on-chip L2 buffers and routing hubs. 8 shim tiles (row 0) provide DMA access to host DDR memory.

Property	Value
Compute tiles	32 (8 columns × 4 rows)
Memory tiles	8 (512 KB each, 4 MB total)
Per-tile SRAM	~64 KB data memory
Per-tile compute	bf16 MMUL unit, VLIW+SIMD core
Clock frequency	~1.5 GHz
Peak throughput	**25 TFLOPS** (bfloat16)
Interconnect	ObjectFIFOs (tile-to-tile double-buffered streams)
Power envelope	~6 W

3.1 Why spatial dataflow matters

On a CPU, a matrix multiply reads data from DRAM through multiple cache levels. Each layer of a neural network bounces activations through L1→L2→L3→DRAM and back. On the NPU, data stays in 64 KB tile SRAM between operations — zero cache misses, zero bus contention. This is the source of the NPU's advantage for deep, narrow computations.

> **Why on-chip matters:** A single large GEMM achieves only 2.49 TFLOPS on the NPU (10% of peak) because it is *memory-bandwidth limited* — data must stream from DDR. The NPU wins when data **stays on-chip**. Our architecture keeps weights in tile SRAM and passes activations tile-to-tile through FIFOs.

3.2 Inside an AIE tile

Each of the 32 compute tiles is a self-contained processor with four main subsystems. Understanding what lives inside a tile is essential for knowing what computation is practical at the single-tile level.

VLIW+SIMD core

The heart of each tile is a VLIW processor that can issue multiple operations per clock cycle: a vector multiply, a vector add, a scalar operation, and a memory load/store — all in the same cycle. The vector unit operates on **256-bit vectors**: 16 elements of bfloat16, 32 elements of int8, or 8 elements of float32. This SIMD width determines the natural “chunk size” for all tile-level programming: data buffers must be aligned to 64 bytes and sized in multiples of 32 bfloat16 elements (64 bytes) for efficient access.

Matrix multiply (MMUL) unit

Dedicated hardware multiplies small matrix tiles in a single instruction. The MMUL unit operates on blocks of size `r×s×t`, meaning it multiplies an `r×s` sub-matrix of A by an `s×t` sub-matrix of B and accumulates into an `r×t` result. For bfloat16 on XDNA 2:

- **Native bf16:** (r, s, t) = (4, 8, 8) — 256 multiply-accumulate operations per instruction.
- **BFP16 emulation:** (r, s, t) = (8, 8, 8) — 512 MACs per instruction, double the throughput. This groups bf16 values into blocks sharing an exponent, trading negligible precision for 2× throughput. We use this mode throughout.

At ~1.5 GHz, a single tile can sustain 512 MACs × 2 FLOPs/MAC × 1.5 GHz ≈ **~768 GFLOPS** peak in BFP16 mode. Multiplied by 32 tiles, this gives the advertised 25 TFLOPS system peak.

How the MMUL instruction works

A single `aie::mmul<8,8,8>::mac(A_tile, B_tile)` call tells the hardware: "take an 8×8 sub-block of A and an 8×8 sub-block of B, compute 512 multiply-accumulates, and add the results into the 8×8 accumulator C." The accumulator is a set of **registers inside the tile core** — no SRAM access needed during the MAC.

To multiply a full matrix C = A × B where A is M×K and B is K×N, the kernel breaks the problem into three nested loops over these r×s×t blocks:

```
Full matmul:  C[M,N] = A[M,K] × B[K,N]     with r=8, s=8, t=8

For each row-block  z = 0, 2, 4, ... M/r-1  (steps of 2, "2×2 expansion")
  For each col-block j = 0, 2, 4, ... N/t-1  (steps of 2)
    Zero 4 accumulators: C00, C01, C10, C11    (each 8×8 = 64 values)
    For each k-block i = 0, 1, ... K/s-1
      Load A0 = A[z·r .. (z+1)·r , i·s .. (i+1)·s]       (8×8 tile)
      Load A1 = A[(z+1)·r .. (z+2)·r , i·s .. (i+1)·s]   (8×8 tile)
      Load B0 = B[i·s .. (i+1)·s , j·t .. (j+1)·t]       (8×8 tile)
      Load B1 = B[i·s .. (i+1)·s , (j+1)·t .. (j+2)·t]   (8×8 tile)
      C00.mac(A0, B0)     ← 512 MACs in 1 instruction
      C01.mac(A0, B1)
      C10.mac(A1, B0)
      C11.mac(A1, B1)     ← 4 × 512 = 2048 MACs per inner iteration
    Store C00, C01, C10, C11 to SRAM (with ReLU fused: max(C, 0))
```

Where the dimension constraints come from

The three `static_assert` lines in our kernel are **software constraints from our kernel design**, not fundamental hardware limitations. It is important to distinguish three layers:

Layer	Constraint	Can be relaxed?
Hardware (MMUL instruction)	Operates on 8×8×8 blocks. Input tiles must be exactly 8×8 elements.	No — this is the physical instruction size. But you can always **pad** your matrices with zeros to the next multiple of 8. IRON's GEMM operator does this transparently.
2×2 expansion (our kernel pattern)	`DIM_M % (2r) == 0` ⇒ B % 16 == 0 `DIM_N % (2t) == 0` ⇒ H % 16 == 0	**Yes.** The 2×2 expansion processes two row-blocks and two column-blocks per iteration for better register utilisation. A simpler

		1×1 kernel would only need B % 8 == 0 and H % 8 == 0.
Pipelining (compiler optimisation)	`chess_prepare_for_pipelining` needs ≥3 outer-loop iterations for effective overlap of load/compute/store.	**Yes.** Fewer iterations still work, just slower. With 2×2: B/(2×8) ≥ 3 ⇒ B ≥ 48. With 1×1: B/8 ≥ 3 ⇒ B ≥ 24.

DIM_M is the batch dimension B (not the hidden size). It appears in the compile command as `-DDIM_M=48`. The matmul computes C[B, H] = A[B, H] × W[H, H]: rows of A correspond to independent samples in the batch.

So B=1 is not impossible in principle — you could pad to B=8 and discard 7 rows of the output. But 7/8 of the MMUL work would be wasted. Similarly, B=8 with a 1×1 kernel would work correctly but with no loop pipelining. The constraints compound: B=48 with 2×2 expansion is the *sweet spot* that simultaneously maximises register utilisation, enables pipelining, and fits in SRAM.

Why B=48 in practice

Even though the hardware could process B=8 or B=24, the throughput difference is dramatic. The `chess_prepare_for_pipelining` directive asks the compiler to overlap iterations: while one iteration's MAC results are being stored to SRAM, the next iteration's A and B tiles are being loaded.

Batch size	Outer loop iters	Pipelined?	Measured TFLOPS
B=8 (padded, 1×1)	1	No	< 4 (estimated)
B=16 (2×2)	1	No	8.04
B=24 (1×1)	3	Yes	~12 (estimated)
B=48 (2×2)	**3**	**Yes**	**23.93 (95.7% peak)**

The 2×2 expansion at B=48 wins because it processes 2048 MACs per inner-loop body (4 accumulators × 512 MACs), keeping all the accumulator registers busy. A 1×1 kernel at B=24 pipelines correctly but only uses one quarter of the register file per iteration.

Tile internals diagram (logical view of one AIE compute tile):

```
┌──────────────────────────────────────────────────────────────┐
│                      AIE Compute Tile                        │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  VLIW + SIMD Core (1.5 GHz)                          │    │
```

```
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐  │  │
│  │  │ Scalar unit  │ │ 256-bit SIMD │ │ MMUL 8×8×8       │  │  │
│  │  │ (control,    │ │ (16 bf16 ops │ │ 512 MACs/instr   │  │  │
│  │  │  loop cntrs) │ │  per cycle)  │ │ 1024 FLOPs/cycle │  │  │
│  │  └──────────────┘ └──────────────┘ └──────────────────┘  │  │
│  │        │              │              │                  │  │
│  │  ┌─────┴──────────────┴──────────────┴──────────────┐  │  │
│  │  │          Register file (256-bit vectors)          │  │  │
│  │  └──────────────────────────────────────────────────┘  │  │
│  └────────────────────────────────────────────────────────┘  │
│                        │ ▲                                   │
│                   load ▼ │  store                             │
│  ┌────────────────────────────────────────────────────────┐  │
│  │ DATA SRAM (~64 KB)                                     │  │
│  │                                                        │  │
│  │  ┌──────────────┐  ┌──────────┐  ┌──────────┐        │  │
│  │  │ W: 128×128   │  │ A_in: buf │  │ C_out: buf │      │  │
│  │  │ + scale      │  │ 48×128   │  │ 48×128   │        │  │
│  │  │ 32.25 KB     │  │ 12 KB    │  │ 12 KB    │        │  │
│  │  └──────────────┘  └──────────┘  └──────────┘        │  │
│  │   (stays)          (from prev)    (to next tile)      │  │
│  └────────────────────────────────────────────────────────┘  │
│                  ▲                        │                  │
│             DMA  │                   DMA  ▼                  │
│  ┌────────────────────────────────────────────────────────┐  │
│  │ DMA Engine (programmed by ObjectFIFO buffer descs)     │  │
│  └────────────────────────────────────────────────────────┘  │
│        ▲ from prev tile                  │ to next tile      │
└────────┼──────────────────────────────────┼─────────────────┘
```

Local SRAM

Each tile has ~**64 KB** of data SRAM — *all* the memory the core can access directly. There are no caches; every byte is explicitly managed by the programmer. A separate, smaller instruction memory holds the compiled kernel. The 64 KB budget must fit: (1) weight data, (2) input and output activation buffers, and (3) stack and local variables. This is the single most important constraint in NPU architecture co-design.

Per-tile DMA engine

Each tile has its own DMA engine that moves data between tile SRAM and neighbouring tiles or the MemTile. The programmer does not call DMA functions directly; instead, IRON's ObjectFIFOs abstract these transfers as typed channels. The compiler generates buffer descriptors that program the DMA with source addresses, sizes, and stride patterns.

> **Mental model:** Think of each tile as a tiny computer with 64 KB of RAM, a fast matrix-multiply co-processor, and a mail slot (DMA/FIFO) for sending and receiving data from

neighbours. There is no shared memory and no operating system — just your compiled C++ function running in an infinite loop, acquiring data, computing, and releasing results.

3.3 Writing AIE kernels (the C++ API)

Each tile runs a C++ function compiled by the AIE-specific Peano/LLVM-AIE compiler. The `aie_api/aie.hpp` header provides intrinsics that map directly to hardware instructions. This section walks through the key patterns using our actual `norm_matmul_relu.cc` kernel as an example.

Vector types

The fundamental data type is a fixed-width SIMD vector:

```
aie::vector<bfloat16, 16>   // 16 × bf16 = 256 bits (one vector register)
aie::vector<int8, 32>       // 32 × int8  = 256 bits
aie::vector<float, 8>       //  8 × f32   = 256 bits
```

Load and store 16 bfloat16 elements at once with `aie::load_v<16>(ptr)` and `aie::store_v(ptr, vec)`. Element-wise operations like `aie::max(vec, zeros)` (ReLU) operate on full vectors.

Matrix multiply: `aie::mmul`

The core workhorse is the `aie::mmul<r, s, t>` class, which wraps the MMUL hardware unit:

```
using MMUL = aie::mmul<8, 8, 8, bfloat16, bfloat16, accauto>;

MMUL acc(zeros);                      // zero-initialise accumulator in registers
auto A = aie::load_v<MMUL::size_A>(pA);   // load 8×8 A tile (64 bf16)
auto B = aie::load_v<MMUL::size_B>(pB);   // load 8×8 B tile (64 bf16)
acc.mac(A, B);                        // acc += A × B  (hardware MAC instruction)
auto result = acc.to_vector<bfloat16>();    // extract 8×8 result as bf16
aie::store_v(pC, aie::max(result, zeros));  // store with fused ReLU
```

The `accauto` type uses higher-precision accumulators internally (typically 48-bit or fp32), converting back to bfloat16 only at `to_vector`. This preserves numerical quality during large reductions.

2×2 tile expansion

Processing one 8×8 block at a time under-utilises registers. Our kernels process **four** output blocks per inner-loop iteration (a 2×2 grid of MMUL accumulators). This keeps 4 accumulator registers active simultaneously, allowing the VLIW scheduler to interleave loads and multiplies:

```
// Four accumulators for a 2×2 grid of output blocks
MMUL C00(zeros), C01(zeros), C10(zeros), C11(zeros);

for (i = 0; i < K/8; ++i) {
    A0 = load(pA1);  A1 = load(pA2);     // two rows of A
    B0 = load(pB1);  B1 = load(pB2);     // two columns of B
    C00.mac(A0, B0);  C01.mac(A0, B1);   // top row of output
    C10.mac(A1, B0);  C11.mac(A1, B1);   // bottom row of output
}
// Store with fused ReLU: max(result, 0)
store(pC, max(C00.to_vector(), zeros));
// ... C01, C10, C11 similarly
```

This pattern yields roughly 2× throughput over naive single-block processing, because the compiler can pipeline loads with multiplies across the four independent accumulations.

Compiler hints

Two pragmas are critical for performance:

- `chess_prepare_for_pipelining` — tells the "chess" scheduler (the AIE backend compiler) to overlap iterations of the loop body, like a CPU's instruction pipeline but at the loop level. Without this, each iteration starts only after the previous one fully completes.
- `chess_flatten_loop` — fully unrolls the inner loop so the compiler can schedule all operations as one large block. Used on tight inner loops where the iteration count is small and known at compile time.

The fused norm+matmul+ReLU kernel

Our `norm_matmul_relu.cc` kernel performs three operations in a single tile invocation:

1. **RMSNorm in-place** (scalar float32): For each of the B rows, accumulate the sum of squares, compute the inverse RMS using 8 Babylonian iterations (avoiding library dependencies), and scale each element by the learned scale vector. Cost: ~650 scalar float ops per row × B rows ≈ 25 µs at M=48, K=128.
2. **Matmul + ReLU** (vectorised bf16): The 2×2 tile expansion described above, with ReLU fused into the store step. At the end: `store(max(acc.to_vector(), zeros))`.

The key implementation detail: weights and scale share one buffer. The weight matrix (H×H = 32 KB in bf16 tiled layout) occupies the first H×H elements, and the scale vector (H elements = 256 bytes) is appended after it. The kernel extracts the scale pointer:

```
const bfloat16 *scale = w_and_scale + DIM_K * DIM_N;
rms_norm_inplace(input, scale);    // step 1: normalise in-place
matmul_relu_2x2(input, w, output); // step 2: matmul + ReLU
```

3.4 IRON: the Python programming model

IRON is a Python API that lets you describe *what* each tile computes and *how* data moves between tiles, without writing any MLIR or assembly by hand. The IRON compiler translates your Python program into MLIR-AIE, then into the binary bitstream (`.xclbin`) and instruction sequence (`.bin`) that configure the hardware.

`ObjectFifo` : explicit data channels

The most important abstraction. An ObjectFifo is a hardware queue between a producer tile and a consumer tile:

```
fifo = ObjectFifo(buffer_type, name="my_fifo", depth=1)
```

The `depth` is how many buffers are in the queue (depth=1 means single-buffering; depth=2 gives double buffering). Each end has two operations:

- `fifo.prod()` / `fifo.cons()` — get the producer or consumer endpoint.
- `acquire(n)` / `release(n)` — **blocking** acquire (wait until n buffers are available) and release (signal the other end that n buffers are done). This is the *only* synchronisation mechanism between tiles.

Three routing operations handle the MemTile as an on-chip switch:

- `.forward()` — pass data through MemTile unchanged (DDR input → MemTile → compute tile).
- `.split(offsets, types)` — one large buffer arriving at MemTile is sliced into N smaller buffers distributed to N tiles. Used for distributing per-stage weights from a single DDR transfer.
- `.join(offsets, types)` — the reverse of split: N tiles' outputs are gathered into one large buffer for DDR.

ObjectFIFO ⇄ hardware mapping: Each ObjectFifo compiles to a pair of buffer descriptors (BDs) in the DMA engines of the producer and consumer tiles. The BDs describe the memory addresses, sizes, and stride patterns. The `acquire/release` calls become hardware lock operations: a tile stalls until the BD signals data is ready. There is zero software overhead at runtime.

`Kernel`: compiled C++ function reference

```
mm_relu = Kernel(
    "norm_matmul_relu_bf16_bf16",   # C function name (extern "C")
    "mlp_kernels.a",                # compiled library
    [act_type, weight_type, act_type]  # argument types
)
```

A Kernel object is just a *reference* to a compiled C++ function. It tells IRON what to link onto each tile. The argument types must match the ObjectFifo buffer types.

`Worker`: one function per tile

```
def stage_body(of_in, of_out, of_w, mm_relu_fn):
    x = of_in.acquire(1)       # wait for input
    y = of_out.acquire(1)      # wait for output buffer
    w = of_w.acquire(1)        # wait for weights
    mm_relu_fn(x, w, y)        # compute: y = ReLU(RMSNorm(x) @ W)
    of_w.release(1)
    of_in.release(1)
    of_out.release(1)          # signal output ready

Worker(stage_body,
       fn_args=[in_fifo.cons(), out_fifo.prod(), wt_fifo.cons(), mm_relu],
       placement=Tile(col=0, row=2))
```

A Worker binds a Python function to a specific physical tile. The function describes the tile's behaviour: acquire inputs from FIFOs, call the kernel, release outputs. IRON compiles this into the tile's instruction sequence. Each tile gets exactly one Worker.

`Runtime`: host–device orchestration

The Runtime describes what the host CPU does: load data into the NPU, start the workers, and drain results back:

```
rt = Runtime()
with rt.sequence(input_type, weight_type, output_type) as (inp, wts, out):
```

```
    rt.start(*workers)          # launch all 32 workers (they block on FIFOs)
    tg = rt.task_group()
    for col in range(8):
        rt.fill(ddr_in[col].prod(), inp, tap, task_group=tg)   # DMA: host → tile
    rt.finish_task_group(tg)
    # ... fill weights, drain outputs similarly
```

`task_group()` groups multiple DMA transfers that execute in parallel (e.g., all 8 columns fill simultaneously). `TensorAccessPattern` (TAP) objects describe how to slice the host buffer into per-column chunks:

```
tap = TensorAccessPattern(
    (1, total_size),    # shape of the host buffer
    col * B * H,        # offset: start of this column's slice
    [1, 1, B, H],       # logical shape for DMA iteration
    [0, 0, H, 1],       # strides
)
```

`Program`: putting it all together

```
program = Program(NPU2(), rt)
mlir_module = program.resolve_program(SequentialPlacer())
```

`NPU2()` selects the XDNA 2 hardware target (8 columns, 4 compute rows). `SequentialPlacer()` assigns Workers to tiles in order. The `resolve_program` call generates the complete MLIR-AIE module, which is then compiled to the final `.xclbin` bitstream and `.bin` instruction file.

3.5 Design patterns for the NPU

With the hardware and programming model understood, here are the key patterns for mapping computation graphs onto the tile array.

Pattern 1: Column parallelism (data-parallel batching)

All 8 columns are identical and independent: they run the same kernel on different slices of the input batch. If you have B samples per column, the total batch is 8×B. This is pure data parallelism — no communication between columns. Each column has its own set of ObjectFIFOs and its own weight copy in tile SRAM.

When to use: Always, for throughput. 8 columns × B samples per column amortises the ~120 µs XRT invocation overhead across 8×B results.

Pattern 2: Row pipelining (sequential stages)

Within each column, the 4 compute rows form a pipeline. Data flows from row 2 → row 3 → row 4 → row 5 through tile-to-tile ObjectFIFOs (no MemTile hop, zero-copy). Each row applies a different transformation (different weight matrix). The pipeline behaviour emerges naturally from FIFO blocking: row 3's `acquire()` stalls until row 2's `release()` signals output is ready.

When to use: For deep, sequential computation (stacked layers). Each pipeline stage adds one more learnable transformation without any DDR round-trip.

Pattern 3: Weight-stationary execution

The weight matrix stays in tile SRAM for the entire NPU invocation. Only activations stream through the tile (one input buffer in, one output buffer out). This is the optimal strategy when:

- The weight matrix fits in SRAM (our 128×128 matrix = 32 KB < 64 KB per tile).
- The same weights are applied to many input samples (batch processing).

The alternative (activation-stationary, streaming weights) is needed when weights exceed tile SRAM — but then the operation becomes memory-bandwidth limited and the NPU loses its advantage over the CPU.

Pattern 4: SRAM budget planning

Every design starts with this calculation. For our architecture (H=128, B=48, BFP16 mode):

Component	Size (bytes)	Notes
Weight matrix W	128×128×2 = 32,768	bf16, 8×8 tiled layout
Scale vector	128×2 = 256	bf16, flat (appended to W)
Input activation buffer	48×128×2 = 12,288	bf16, one batch
Output activation buffer	48×128×2 = 12,288	bf16, one batch
Stack + code locals	~1,500	Estimated
Total	**~59,100 (57.7 KB)**	**Fits in 64 KB ✓**

This budget leaves ~6 KB of headroom. Increasing H to 192 would need 192×192×2 = 72 KB for weights alone — over the 64 KB limit. This is why H=128 is the maximum hidden dimension for single-tile weight-stationary execution in bf16.

Pattern 5: Kernel fusion

Unfused operations (separate kernels for norm, matmul, and relu) require three DMA round-trips per stage, three kernel invocations, and three sets of buffers. Fusing them into one kernel:

- Eliminates intermediate buffers (input is normalised in-place).
- Keeps data in registers between operations.
- Reduces the number of Worker `acquire/release` cycles.
- Allows the compiler to pipeline the norm reduction with the matmul loads.

In our benchmarks, the fused `matmul_relu` kernel achieved **23.93 TFLOPS** (95.7% of peak) versus 15.98 TFLOPS unfused at B=48 — a 50% throughput improvement from fusion alone.

Pattern 6: Amortising invocation overhead

Every NPU call has ~120 µs of XRT driver overhead. If each call does only one matmul (~1 µs of compute), you waste 99% of time on overhead. Solutions:

- **Large batches:** B=48 means 48 rows of 128 multiplied per tile, 24× more work per call than B=2.
- **Deep pipelines:** 4 stages per call means 4 matmuls of compute for one overhead.
- **Hardware loops:** For throughput benchmarks (not autoregressive generation), a `range_(N)` loop repeats the computation N times *on-chip* before returning to the host. At N=1000, compute completely dominates overhead.

3.6 Hardware constraints and gotchas

The NPU has several constraints that are not obvious from high-level descriptions. We discovered each of these through trial and error during development.

Buffer descriptor size limit (10-bit fields)

The DMA engines in shim tiles use buffer descriptors with 10-bit size fields, meaning each dimension of a `TensorAccessPattern` must be ≤ 1024. If your tensor has a dimension larger than 1024 (e.g., B×H = 48×128 = 6144), you must factor it: use shape `[tiles_per_col, B, H]` instead of `[tiles_per_col, B*H]`. The compiler will reject designs that violate this, with the cryptic error `“Size 0 exceeds [0:1023] range”`.

MemTile routing limits

Each MemTile has approximately 6 master ports going northward (toward compute rows). A multi-row design that uses `split()` + `forward()` + `join()` consumes 3 FIFOs per row of compute tiles. This means:

- **3 compute rows** (24 tiles) works reliably: 3×3 = 9 FIFOs, within limits.
- **4 compute rows** (32 tiles) sometimes fails with `"Unable to find a legal routing"`, depending on the FIFO topology.

Our design uses all 4 compute rows by routing inter-tile activations *directly tile-to-tile* (not through MemTile), reserving MemTile ports for DDR-facing traffic only (weight split, activation forward/join).

ShimDMA channel limits

There are 16 ShimDMA channels total (2 per column). Each DDR-facing ObjectFifo consumes one channel. With 8 columns, our design uses 3 DDR FIFOs per column (input, weight, output) = 24, but the compiler shares channels across task groups that don't overlap in time. Exceeding 16 *simultaneous* channels causes compilation failure.

ObjectFIFO acquire/release inside loops

Placing `acquire()` and `release()` calls inside an IRON `range_(N)` hardware loop can cause deadlocks: the tile's DMA and compute core enter a circular dependency where each waits for the other. The fix: acquire all FIFOs *before* the loop, put only kernel calls inside, and release *after* the loop. This was the most subtle bug we encountered and caused silent hardware timeouts (`ERT_CMD_STATE_TIMEOUT`) with no diagnostic output.

Tiled memory layout

The MMUL hardware expects weight matrices in **blocked (tiled) format**: the matrix is divided into 8×8 sub-matrices stored contiguously, rather than row-major order. A 128×128 matrix becomes 16×16 blocks of 64 elements each. Host-side code must convert with `to_tiled()` before sending weights to the NPU, and `from_tiled()` to interpret results. Forgetting this conversion produces silently wrong results, not errors.

3.7 Putting it all together

Let's trace how these pieces combine to build a complete NPU design, starting small and scaling up.

Step 1: Single-tile matmul

The simplest possible design: one tile, one weight matrix, one input, one output. Two ObjectFIFOs (in, out), one Kernel, one Worker. The host fills the input FIFO, the tile computes `y = ReLU(x @ W)`, and the host drains the output. This uses 1 of 32 tiles and achieves ~1/32 of peak throughput, but it validates the kernel and data layout.

Step 2: Column parallelism (8 tiles)

Replicate the single-tile design across 8 columns. Each column gets its own ObjectFIFOs and Worker, processing a different batch slice. The Runtime uses `task_group()` to issue 8 parallel DMA fills. Throughput scales linearly: 8× the single-tile result.

Step 3: Row pipelining (4×8 = 32 tiles)

Add 3 more rows per column. Inter-tile FIFOs chain the stages: row 2's output feeds row 3's input, and so on. Weights are distributed via `.split()` through the MemTile. Now each NPU call applies 4 sequential transformations — equivalent to 4 layers of a neural network.

Step 4: Host-orchestrated depth (8 blocks × 4 stages = 32 layers)

The host calls the NPU 8 times, each time loading a different set of 4 weight matrices. Between calls, the CPU applies normalisation and residual connections. The result is a 32-layer model running on 32 tiles, with the architecture described in the next section.

> **Design methodology:** Start with one tile, get it working, then scale horizontally (columns) for throughput and vertically (rows) for depth. Each scaling step adds hardware resources without changing the per-tile kernel. The host orchestration layer (IRON Runtime) manages the increasing complexity.

5. The Architecture: Block-Recurrent Character LM

We build a character-level language model whose architecture is dictated by the NPU's physical tile array. The model has 32 layers (one per compute tile), grouped into 8 blocks of 4 layers (one block per column). Each block maps to a single NPU pipeline call. Between blocks, the CPU applies the operations that the pipeline cannot: normalisation, embedding injection, and residual connections.

5.1 The block-recurrent computation

For each character in the input sequence, the model updates a hidden state `h` (a 128-dimensional bfloat16 vector) through 8 blocks:

```
for each character:
    for each block g = 0..7:
        CPU:  h_b = h + embed(char) + bias_g              ← inject input
        NPU:  for stage j = 0..3:
                  h_b = ReLU(RMSNorm(h_b) @ W[4g+j])      ← fused on-chip
        CPU:  h = h + h_b                                  ← residual connection
    CPU:  h = RMSNorm(h)                                   ← post-norm
    CPU:  logits = h @ W_out + b_out                       ← predict next char
```

Each of the 8 NPU calls runs a 4-stage pipeline where each stage fuses RMSNorm + matmul + ReLU:

1. The CPU prepares the block input: add the character embedding and a per-block learned bias to the hidden state (two vector additions).
2. The prepared vector is sent to the NPU, where it passes through 4 tiles in sequence. Each tile normalises its input (RMSNorm), multiplies by its own weight matrix W_i (128×128), and applies ReLU — all in a single fused `norm_matmul_relu` kernel.
3. The NPU returns the result to the CPU, which adds it back to the hidden state (residual connection).

After all 8 blocks, a final RMSNorm and a linear readout produce a probability distribution over the ~65-character vocabulary.

5.2 NPU tile mapping

The model uses all 32 compute tiles in an 8-column × 4-row layout:

```
Column 0        Column 1        ...  Column 7
(samples 0-47)  (samples 48-95)      (samples 336-383)

  Row 2: W&sub0;       Row 2: W&sub0;    ...    Row 2: W&sub0;      Stage 1
  ↓               ↓                    ↓
  Row 3: W&sub1;       Row 3: W&sub1;    ...    Row 3: W&sub1;      Stage 2
  ↓               ↓                    ↓
  Row 4: W&sub2;       Row 4: W&sub2;    ...    Row 4: W&sub2;      Stage 3
  ↓               ↓                    ↓
  Row 5: W&sub3;       Row 5: W&sub3;    ...    Row 5: W&sub3;      Stage 4
```

- **8 columns** process 8 independent batch slices in parallel, 48 samples each = 384 total.

- **4 rows** per column form a pipeline: data flows from row 2 to row 5 through ObjectFIFOs, tile-to-tile, with no DDR traffic between stages.
- All 8 columns share the same 4 weight matrices for a given block. The CPU loads the appropriate block's weights before each NPU call.

5.3 Why this architecture

Every architectural choice follows from a hardware constraint:

- **H=128:** A 128×128 weight matrix plus a 128-element RMSNorm scale vector occupies 32.25 KB in bf16. With two activation buffers (48×128 = 12 KB each, double-buffered) and ~1.5 KB of stack+code, the total is ~58 KB — within the 64 KB per-tile SRAM limit.
- **4 layers per block:** The NPU has 4 compute rows. A 4-stage pipeline uses one row per stage, fully utilizing the vertical dimension.
- **8 blocks:** With 4 layers per block and 32 total layers, 8 blocks = 32 layers = 32 tiles. This is the maximum depth that maps cleanly to the hardware.
- **B=48:** The fused `matmul_relu` kernel pipelines efficiently only when the outer loop has ≥3 iterations: M/(2r) ≥ 3, so B ≥ 48 for r = 8. (See `logbook.md` for the kernel fusion analysis.)
- **384 parallel sequences:** 8 columns × 48 samples per column. Processing many sequences in parallel amortises the ~120 µs XRT driver overhead per NPU call.

Why not B=1 with a larger hidden dimension?

A natural question: could each column process a *single* sequence (B=1) and use the freed SRAM for a larger weight matrix?

B=1 is *not* a hardware impossibility. The MMUL instruction needs 8×8 input tiles, so B=1 would require padding to B=8 (7 zero rows). This is valid — IRON's GEMM operator does exactly this for arbitrary dimensions. But it is highly wasteful:

1. **7/8 wasted compute:** With B=1 padded to 8, the MMUL processes 8 rows but only 1 produces useful output. One matmul yields 32K useful FLOPs instead of 1.57M with B=48 — a **48× efficiency loss**.
2. **No loop pipelining:** The outer loop has only 1 iteration (B/8=1), so the chess compiler cannot overlap load/compute/store phases. Peak throughput drops from 23.93 to <4 TFLOPS.
3. **SRAM still limits H:** Even with B=1, the weight matrix alone is H×H×2 bytes. H=192 needs 72 KB > 64 KB. The maximum reachable is H≈176 — only 37% more parameters per layer, while losing 48× the batch throughput.

At B=1, a CPU is equally fast or faster — the NPU's advantage comes precisely from processing many rows in parallel through its systolic array.

An alternative approach would be to *shard* a large weight matrix across multiple tiles (e.g., H=512 split into four 512×128 slices). This requires horizontal communication between tiles for partial-sum reduction, which the current column topology does not support. Exploring such *model-parallel* layouts is future work.

5.4 Fusing normalisation into the pipeline

The key innovation is a custom NPU kernel that fuses three operations — RMSNorm, matrix multiply, and ReLU — into a single per-tile function. Without this fusion, normalisation would require CPU intervention between every pipeline stage, defeating the purpose of the 4-stage streaming pipeline.

The fused `norm_matmul_relu` kernel for each pipeline stage:

1. **RMSNorm in-place** (scalar float32 for stability): compute the root-mean-square of the input row, divide each element by it, and multiply by the learned scale vector. This is a reduction over 128 elements per row, using 8 Babylonian iterations for the inverse square root to avoid library dependencies.
2. **Fused matmul + ReLU** (vectorised bf16): the normalised input feeds directly into the same 2×2 tile-expanded matrix multiply used previously, with ReLU applied during the store phase.

The scale vector (128 × 2 bytes = 256 bytes) is packed at the end of each tile's weight buffer: [W (32 KB), scale (256 B)]. This adds only 0.25 KB per tile, well within the SRAM budget.

Moving per-layer normalisation into the NPU pipeline closed the quality gap from val loss 2.42 (pure matmul+ReLU blocks) to 2.03, approaching the per-layer CPU-only model's 1.94 — while keeping the full 32-layer inference on the NPU.

Design principle: We train the model with exactly the same block structure that runs on the NPU. There is no "compilation" step that approximates a richer model for hardware. What you train is what you deploy.

5.5 Making it trainable

Simply stacking 32 layers of ReLU(h × W + b) does not work: the hidden state either explodes to infinity or collapses to zero. We use three techniques (explained in Section 2.7):

- **Per-stage RMSNorm:** Each pipeline stage normalises its input to unit RMS *before* the matrix multiply. Because the norm is fused into the NPU kernel, this happens on-chip with no CPU intervention. This prevents the activations from growing exponentially through the 4-layer chain and across the 64-character sequence during backpropagation through time.
- **Residual connections:** `h = h + block(h)` after each block. Gradients flow backwards through the identity shortcut, keeping training stable even at 32 layers.
- **Input injection:** The character embedding is added at the start of every block, not just the first. This ensures each block can directly access the current input, rather than relying on information that has been progressively transformed by preceding blocks.
- **Spectral norm clamping:** During training, each weight matrix is periodically rescaled so its largest singular value ≤ 1. This prevents any single layer from amplifying the hidden state, keeping the recurrent dynamics stable.

6. Results

6.1 Language model quality

The block-recurrent model was trained on the tiny Shakespeare dataset (1.1 MB, ~1.1M characters) for 10 epochs on a GPU, then deployed to the NPU:

Model	Parameters	Val Loss	Perplexity	Device
Block-recurrent (fused norm+mm+relu)	**542K**	**2.03**	**7.6**	**NPU (32 tiles)**
Per-layer recurrent (norm every layer)	542K	1.94	6.9	CPU/GPU only
Transformer baseline (GPT-style)	818K	1.89	6.6	GPU

The fused norm+matmul+ReLU kernel closes the quality gap: the NPU model (perplexity 7.6) approaches the per-layer CPU model (6.9) and the transformer baseline (6.6), while running entirely on the 32-tile NPU pipeline. The model produces recognisable Shakespearean English with a 542K-parameter model trained on 1 MB of text.

6.2 Inference performance

Throughput comparison (single-sequence autoregressive generation)

Model	Device	chars/s	ms/char
Recurrent 64L (1M params)	CPU	**1,065**	0.94
Transformer 4L (818K params)	CPU	775	1.29
NPU block-recurrent 32L (542K params)	NPU ×1 seq	233	4.29
Transformer 4L (818K params)	GPU	187	5.35
Recurrent 64L (1M params)	GPU	92	10.90

For a single sequence, the CPU is fastest. The models are too small (< 1M parameters) to saturate the GPU's compute units, so GPU kernel launch overhead dominates. The NPU likewise loses on single-sequence latency due to the ~120 µs XRT overhead per call.

NPU batched throughput

Metric	Value
NPU tiles used	32 (8 columns × 4 rows)
NPU calls per character	8 (one per block)
Latency per NPU call	~0.14 ms
Total NPU time per character	~1.1 ms
Total time per character (NPU + CPU)	~4.2 ms
Throughput (1 sequence)	233 chars/s
Throughput (384 parallel sequences)	**89,600 chars/s**

The 384-sequence throughput is the key figure: by processing 8 columns × 48 samples per column in parallel, we amortise the per-call overhead across hundreds of sequences. Compared to the CPU baseline (1,065 chars/s for a single sequence), the NPU achieves **84× higher total throughput** when all 384 sequences are served simultaneously. The CPU overhead between NPU calls (embedding injection, bias addition, residual connection) is minimal since normalisation is now handled on-chip by the fused kernel.

6.3 What limits throughput

Each NPU call performs only 4 matmuls of size 48×128×128 — a tiny amount of arithmetic completed in microseconds. The ~0.14 ms measured per call is dominated by XRT driver overhead (instruction dispatch, DMA setup), not compute. With 8 calls per character, that overhead adds up.

Utilisation arithmetic

Each character step (one character for all 384 sequences) requires:

- 8 NPU calls × 32 tiles × one matmul [48, 128, 128]
- FLOPs per matmul: 2 × 48 × 128 × 128 = 1,572,864
- Total per step: 8 × 32 × 1.57M = **402.7M FLOPs**

At 233 steps/s this is **93.8 GFLOPS — only 0.38% of the 25 TFLOPS bf16 peak**.

Budget item	Time	% of step
Pure MMUL compute (402.7M FLOPs ÷ 23.93 TFLOPS)	~17 µs	0.4%
DMA + instruction dispatch (8 calls × ~0.14 ms)	~1.1 ms	26%
CPU overhead (embedding injection, residuals, softmax)	~3.1 ms	74%
Total per step	**~4.3 ms**	**100%**

The MMUL units are essentially idle. The bottleneck is entirely in the 8 round-trips between CPU and NPU per character. This is fundamentally different from the throughput benchmark (see `logbook.md`), where a single NPU call ran 1000 loop iterations on-chip, making compute dominate overhead and achieving 95.7% of the 25 TFLOPS peak.

> **Insight:** The current architecture proves the hardware mapping is correct but achieves only 0.4% of peak compute utilisation. The path to high utilisation is reducing the number of CPU–NPU round-trips per character: either by running all 8 blocks on-chip in a single NPU call (requires on-chip residual addition and embedding injection), or by switching to INT8 mode (doubling the compute/overhead ratio), or both.

7. The Toolchain

Component	Role

IRON	Python API for tile layout, ObjectFIFOs, and dataflow
MLIR-AIE	MLIR dialect → hardware bitstream compilation
Peano/LLVM-AIE	C++ compiler for per-tile kernels
XRT	Runtime for loading and executing on the NPU

7.1 Compilation pipeline

```
design.py  →  MLIR  →  aiecc  →  .xclbin (bitstream)
                          →  .bin   (instruction sequence)

norm_matmul_relu.cc →  mlp_norm_mm_relu.o →  mlp_kernels.a
mlp_kernels.cc     →  mlp_copy.o         ↗
```

8. Code Structure

The project is intentionally minimal — four Python files and two C++ files:

File	Lines	Purpose
`char_lm/model.py`	~210	Block-recurrent character LM (RecurrentCharLM)
`char_lm/train.py`	~250	GPU training loop (ROCm / CUDA), spectral norm clamping
`char_lm/generate.py`	~215	Text generation on CPU or NPU
`char_lm/data.py`	~80	Shakespeare dataset and vocabulary
`char_lm/transformer_baseline.py`	~240	GPT-style reference model for quality comparison
`spatial_mlp/__init__.py`	~55	Tiling utilities (`to_tiled` , `from_tiled`)
`spatial_mlp/pipeline_design.py`	~290	IRON design: 32-tile pipeline (8 cols × 4 rows)
`spatial_mlp/pipeline_op.py`	~130	IRON operator: compilation + runtime buffers
`aie_kernels/norm_matmul_relu.cc`	~170	Fused C = ReLU(RMSNorm(A, scale) × W) kernel for AIE2P
`aie_kernels/mlp_kernels.cc`	~55	Support kernel: `copy_bf16` for result staging

The character LM module (`char_lm/`) handles training and generation. The spatial MLP module (`spatial_mlp/`) provides the NPU pipeline infrastructure used by the generator for on-device inference.

9. Future Work

- **INT8 mode:** The NPU's peak is 50 TOPS for int8 — double the bf16 rate. With quantization-aware training, this could push effective throughput to ~48 TOPS and allow H=256 (fitting in the same SRAM budget).
- **Vectorised RMSNorm:** The current kernel uses scalar float32 for the normalisation reduction. A vectorised implementation using v16float SIMD could reduce the per-stage norm overhead from ~25 µs to ~3 µs, improving pipeline throughput when compute (not driver overhead) is the bottleneck.
- **Training on NPU:** Research NPU backpropagation (see arXiv:2504.03083). The block structure could allow per-block gradient computation on-chip, with CPU handling the cross-block gradient flow.
- **Larger tasks:** Apply the block-recurrent architecture to longer text datasets, music generation, or time-series forecasting where deep recurrence is natural and the NPU throughput advantage compounds over long sequences.

References

1. AMD IRON repository: github.com/amd/IRON
2. MLIR-AIE programming guide: github.com/Xilinx/mlir-aie
3. IRON tutorial (IPDPS 2025): AMD Technical Paper
4. NPU training: arXiv:2504.03083
5. Linux kernel NPU docs: kernel.org